



Budapesti Műszaki és Gazdaságtudományi Egyetem
Villamosmérnöki és Informatikai Kar
Távközlési és Mesterséges Intelligencia Tanszék

Major Term Project – Final Milestone

SHIP DETECTION IN SATELLITE IMAGERY

Team Name: 99_Problems_but_a_Batch_Aint_One

Bologa Eduárd (Neptun: DAM4AV)

Kozma Szabolcs András (Neptun: TKGQWN)

Püinkösti Györk (Neptun: VCV3N5)

VITMAV45 – *Deep Learning in Practice*

with Python and LUA

BUDAPEST, 2025

Tartalomjegyzék

1. Introduction	3
2. Problem domain and related work.....	3
2.1 Ship detection in satellite imagery	3
2.2 Class imbalance and modelling challenges	4
2.3 Deep learning approaches for ship detection.....	4
3.1 Stage 1: Binary classification	6
3.2 Stage 2: Ship segmentation	6
3.3 Inference Logic and design rationale	6
4. Data acquisition, exploration and preparation.....	7
5. Training setup and experimental environment	7
6. Training procedure and optimization	7
7. Evaluation and results.....	8
Irodalomjegyzék	Error! Bookmark not defined.
Függelék.....	Error! Bookmark not defined.

1. Introduction

The objective of this project is to design and deploy a deep learning–based system for ship detection in satellite imagery (Cheng et al., 2016; Li et al., 2017) as the major term project of the course VITMAV45 – Deep Learning in Practice with Python and LUA. The task was selected to demonstrate the complete deep learning workflow, from data preparation and model training to evaluation and deployment.

We chose the Airbus Ship Detection Challenge (Van Etten et al., 2018) because it represents a realistic segmentation problem with extreme class imbalance (Lin et al., 2017), where most images contain only sea. This characteristic introduces practical challenges such as false positive detections and inefficient inference, making the dataset well suited for exploring real-world limitations of standard deep learning approaches. To address these issues, we implemented a two-stage pipeline consisting of a binary classifier followed by a high-resolution segmentation model. The classifier filters out empty images, while the segmentation network performs pixel-level ship detection on selected inputs. This design reduces false positives, improves computational efficiency, and aligns optimization with the recall-focused F2 evaluation metric.

To ensure practical applicability, the final solution was deployed as a containerized full-stack web application using FastAPI, React, and Docker, demonstrating how experimental models can be transitioned into reproducible, usable systems.

2. Problem domain and related work

2.1 Ship detection in satellite imagery

Ship detection in satellite imagery is a well-established problem within the fields of remote sensing and computer vision (Cheng et al., 2016; Zhang et al., 2020). Its applications include maritime traffic analysis, illegal fishing detection, search and rescue operations, and national security. Compared to natural image datasets, satellite images introduce unique challenges such as large spatial resolutions, varying illumination and weather conditions, and complex background textures caused by waves, clouds, or coastal regions.

A central distinction in this domain is between object detection and image segmentation. Object detection approaches aim to localize ships using bounding boxes, while segmentation methods perform pixel-level classification, producing more precise ship outlines. The Airbus

Ship Detection Challenge adopts the latter approach, providing segmentation masks encoded using run-length encoding (RLE) (Ronneberger et al., 2015). This makes the task particularly suitable for convolutional encoder–decoder architectures, such as U-Net–based models.

2.2 Class imbalance and modelling challenges

A key challenge of the Airbus dataset is class imbalance, as most images do not contain ships. Without special handling, segmentation models tend to produce visually plausible but incorrect detections on waves or clouds, leading to high false positive rates. Standard loss functions often struggle in this setting, and applying a segmentation model to every image is computationally inefficient (Salehi et al., 2017). These limitations motivate the use of imbalance-aware loss functions and architectural strategies that reduce unnecessary computation on empty images.

2.3 Deep learning approaches for ship detection

Modern ship detection systems are predominantly based on convolutional neural networks, with U-Net–style architectures and pretrained encoders such as ResNet or EfficientNet (He et al., 2016; Tan & Le, 2019) commonly used to improve feature extraction and convergence (Ronneberger et al., 2015). To better handle imbalance, loss functions such as Focal Loss and Tversky Loss are frequently applied, particularly when recall-focused metrics like the F2 score are used for evaluation.

Motivated by these observations, our approach combines a lightweight classifier with a high-resolution segmentation model in a two-stage pipeline. This design reduces false positives, improves efficiency, and aligns model optimization with the evaluation objective, making it well suited for both experimental evaluation and practical deployment.

3. System architecture and model design

The proposed solution follows a two-stage deep learning architecture designed to address extreme class imbalance while maintaining efficient inference. The overall system consists of a lightweight classifier followed by a high-resolution segmentation model, as illustrated in Figure 1.

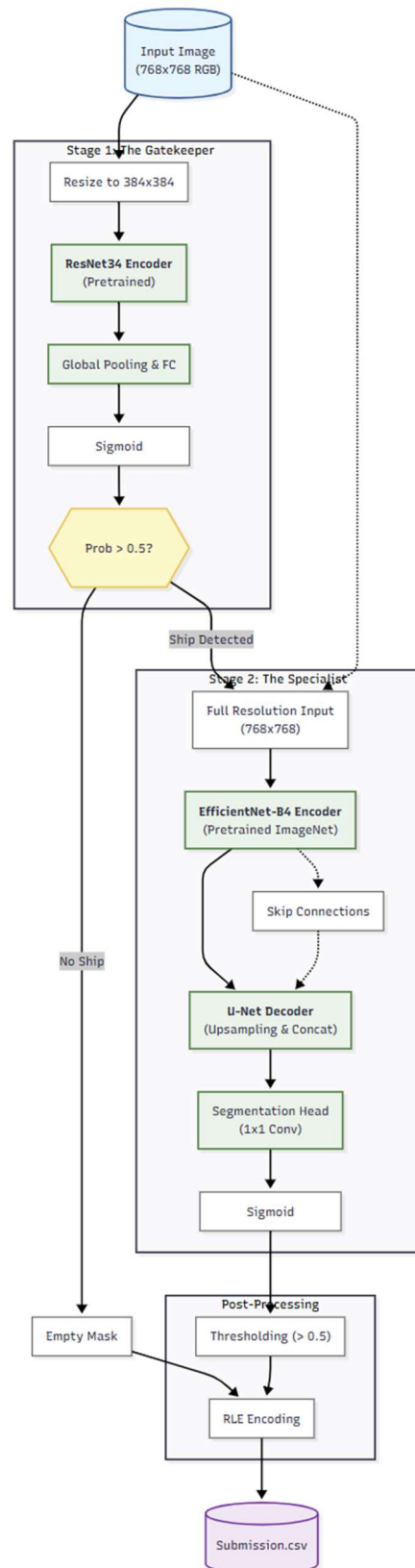


Figure 1: Complete inference workflow

Figure 1 shows the complete inference workflow: an input satellite image is first evaluated by a binary classifier. Only images predicted to contain ships are forwarded to the segmentation stage, which produces a pixel-level ship mask.

3.1 Stage 1: Binary classification

The first stage is a binary image classifier responsible for detecting the presence of at least one ship in an image.

- Architecture: ResNet34
- Task: Ship / no-ship classification
- Input resolution: 384×384 pixels

This stage serves as a computational filter, preventing unnecessary execution of the segmentation model on empty images. Using a lower input resolution allows fast inference while preserving sufficient semantic information for reliable classification. Empirically, this stage filters out approximately 70% of empty images, significantly reducing false positives and computational cost.

3.2 Stage 2: Ship segmentation

Images classified as positive are passed to the second stage, which performs pixel-level segmentation.

- Architecture: U-Net with EfficientNet-B4 encoder
- Input resolution: 768×768 pixels
- Output: Binary segmentation mask

U-Net was selected due to its strong spatial localization capabilities, while the EfficientNet-B4 encoder provides an effective balance between representational power and efficiency. Skip connections enable accurate reconstruction of ship boundaries, which is essential for precise segmentation.

3.3 Inference Logic and design rationale

During inference, the two stages are executed sequentially. If the classifier confidence falls below a predefined threshold, the image is directly labeled as empty. Otherwise, segmentation is applied. This design reduces false positive detections on visually complex

backgrounds such as waves and clouds. The architecture was intentionally designed to be modular, allowing independent training, evaluation, and deployment of each component. This modularity supported incremental development throughout the semester and simplified integration into the deployed backend service.

4. Data acquisition, exploration and preparation

The dataset used in this project comes from the Airbus Ship Detection Challenge on Kaggle and consists of high-resolution satellite images with pixel-level ship annotations encoded using run-length encoding (RLE). Most images contain no ships, resulting in a strongly imbalanced dataset.

During Milestone 1, exploratory analysis was performed to examine image characteristics, mask quality, and ship frequency. Based on these observations, the dataset was split into training, validation, and test sets while preserving the distribution of empty and ship-containing images. Separate labels were prepared for the classification and segmentation tasks.

For data preparation, RLE masks were decoded into binary arrays. To improve performance, RLE strings were pre-processed before dataset iteration, reducing CPU overhead during training. Standard preprocessing and augmentation techniques, including resizing, normalization, and random spatial transformations, were applied to support model generalization.

5. Training setup and experimental environment

All experiments were implemented in Python using PyTorch and conducted in a controlled local environment to ensure reproducibility. Training was performed on a workstation equipped with an NVIDIA RTX 5070 GPU (12 GB VRAM), 64 GB RAM, and a multi-core Intel i9 CPU, enabling efficient training of both the classifier and segmentation models. Exploratory experiments were carried out in Jupyter Notebooks, while finalized training and inference logic was implemented in standalone Python scripts. This setup supported rapid iteration during development and stable execution in the final system.

6. Training procedure and optimization

Model training was carried out incrementally over the course of the semester, following the milestone structure defined for the project. During Milestone 2, an initial U-Net-based segmentation model was trained to establish a baseline and to validate the data pipeline. This

phase focused primarily on training stability, data loading efficiency, and baseline metric evaluation.

In the final milestone, training procedures were refined to support the proposed two-stage pipeline. The classifier and segmentation models were trained independently using their respective datasets and objectives, and the optimized data loading strategy was integrated into both workflows. Post-training, validation-based threshold tuning was applied to calibrate the segmentation outputs for optimal F2 performance.

Rather than introducing new architectural components at this stage, the final training phase emphasized system-level optimization, ensuring that the trained models behaved consistently within the complete inference pipeline and during deployment. These refinements resulted in a more stable, efficient, and evaluation-aligned system.

7. Evaluation and results

The proposed system was evaluated on a held-out test set using the metrics defined by the Airbus Ship Detection Challenge. Particular emphasis was placed on the F2 score (Van Etten et al., 2018), which prioritizes recall and reflects the importance of detecting as many ships as possible. The final two-stage pipeline achieved strong performance on unseen data. The classifier-based filtering stage significantly reduced false positive detections on empty images, while the segmentation model maintained a high recall rate, successfully identifying the majority of real ships. This confirms the effectiveness of separating ship presence detection from pixel-level segmentation.

On the test set, the system achieved an F2 score of 0.8950, with 83.14% precision and 91.24% recall. These results represent a clear improvement over earlier baseline segmentation experiments conducted during the semester and validate the design choices introduced in the final milestone.

References

- Cheng, G., Zhou, P., & Han, J. (2016). Learning rotation-invariant convolutional neural networks for object detection in VHR optical remote sensing images. *IEEE transactions on geoscience and remote sensing*, 54(12), 7405-7415. <https://doi.org/10.1109/TGRS.2016.2601622>
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 770-778).
- Lin, T. Y., Goyal, P., Girshick, R., He, K., & Dollár, P. (2017). Focal loss for dense object detection. In *Proceedings of the IEEE international conference on computer vision* (pp. 2980-2988).
- Liu, Y., Zhang, R., Deng, R., & Zhao, J. (2023). Ship detection and classification based on cascaded detection of hull and wake from optical satellite remote sensing imagery. *GIScience & Remote Sensing*, 60(1), 2196159. <https://doi.org/10.1080/15481603.2023.2196159>
- Ronneberger, O., Fischer, P., & Brox, T. (2015, October). U-net: Convolutional networks for biomedical image segmentation. In *International Conference on Medical image computing and computer-assisted intervention* (pp. 234-241). Cham: Springer international publishing. https://doi.org/10.1007/978-3-319-24574-4_28
- Salehi, S. S. M., Erdogmus, D., & Gholipour, A. (2017, September). Tversky loss function for image segmentation using 3D fully convolutional deep networks. In *International workshop on machine learning in medical imaging* (pp. 379-387). Cham: Springer International Publishing. https://doi.org/10.1007/978-3-319-67389-9_44
- Tan, M., & Le, Q. (2019, May). Efficientnet: Rethinking model scaling for convolutional neural networks. In *International conference on machine learning* (pp. 6105-6114). PMLR.
- Van Etten, A., Lindenbaum, D., & Bacastow, T. M. (2018). Spacenet: A remote sensing dataset and challenge series. *arXiv preprint arXiv:1807.01232*. <https://doi.org/10.48550/arXiv.1807.01232>
- Zhang, C., Shi, S., Xu, X., & Yue, J. (2020). Ship detection in optical remote sensing images based on deep learning. *IEEE Geoscience and Remote Sensing Letters*, 17(5), 781–785. <https://doi.org/10.48550/arXiv.1807.01232>
- Zhao, Z. Q., Zheng, P., Xu, S. T., & Wu, X. (2019). Object detection with deep learning: A review. *IEEE transactions on neural networks and learning systems*, 30(11), 3212-3232. <https://doi.org/10.1109/TNNLS.2018.2876865>

Appendix I

Declaration on the Use of Generative Artificial Intelligence

During the preparation of this project, generative artificial intelligence tools (including ChatGPT and Google Gemini) were used in a supportive role. All AI-assisted content was reviewed, verified, and adapted by the authors, and full responsibility for the final content remains with the project team. Generative AI tools were not used to generate experimental results, train models, or fabricate data. Their use was limited to assisting with documentation, structuring, code readability, and exploratory idea generation.

Usage Type	Generative AI Tool(s)	Affected Parts	Estimated Usage
Background research support	ChatGPT, Gemini	Introduction, Related Work	~25%
Text structuring & outlining	ChatGPT	Overall document structure	~10%
Text refinement & language polishing	ChatGPT	Documentation text	~5%
Code formatting & commenting	ChatGPT, Gemini	Python scripts (formatting and comments)	~25%
Idea generation & development suggestions	ChatGPT, Gemini	Optimization ideas, future work	~15%
UI/UX conceptual assistance	ChatGPT	Web application frontend concepts	~40%
Static analysis & code review support	ChatGPT	Code quality checks	~20%

Generative AI tools were primarily used to support documentation, structural organization, code readability, and exploratory brainstorming. All core technical tasks, including data preprocessing, model training, evaluation, and system implementation, were performed independently by the project team. AI-generated suggestions were applied selectively and critically reviewed before inclusion.